Local LLM Decision Platform for Support-Ticket Triage on Apple Silicon

Subtitle: A technical report on the ADIP/Laya-MLX system

Abstract

Support-ticket triage at small-to-medium scale remains slow, inconsistent, and expensive when routed through hosted LLM APIs. This report describes the Apple Silicon Decision Platform (ADIP), an open-weight, on-premise decision layer that classifies inbound support tickets into a department (billing, technical, sales, or account), assigns an urgency level, and flags refund-related intent — all without egressing customer data. The system is built on the Laya typed-decision model, served locally via the laya-mlx port for Apple MLX, and constrained by a deterministic policy guardrail.

Across a 200-call load test on an Apple M1 Pro (14-core GPU, 16 GiB), the platform delivered a median decision latency of 93.7 ms and a p95 of 256.4 ms — comfortably under the 150 ms / 400 ms targets. On the 50-ticket golden evaluation set, the system passed all 9 strict quality gates: macro-F1 of 0.77 on department routing, expected calibration error (ECE) of 0.098 on department confidence (calibrated), 0.15 on urgency, and 0.037 on refund detection. Operating cost is effectively zero per decision (~$0.000002 amortized hardware cost), with zero network egress and full Apache-2.0 licensing.

Key limitations include Apple-Silicon-only serving, a 512-token English input context (1,024 for multilingual), confidence scores that are not accuracy guarantees (hence the router), and a small evaluation set (n=50) that constrains statistical precision. The platform is designed as a proposal layer — Laya proposes; deterministic policy guardrail decides — and is intended for internal tooling, not high-stakes automated actions.

---

1. Problem and Context

Support teams face a recurring trade-off: cloud LLM APIs (OpenAI, Anthropic, etc.) offer strong general reasoning but introduce per-call cost, data-egress risk, latency variance, and vendor lock-in. For a mid-size SaaS company processing a few hundred to a few thousand tickets per day, the marginal cost of a cloud call is small per ticket but scales linearly, and every ticket body — often containing customer names, order IDs, and diagnostic details — leaves the premises.

Alternative approaches have their own friction. Self-hosting a full generative LLM (e.g., a 7B-parameter model) requires GPU-class hardware, produces free-text output that is hard to validate, and hallucination risk is non-zero even with temperature clamping. Rule-based classifiers and classical ML pipelines (TF-IDF + logistic regression) are cheap and deterministic but require manual feature engineering per domain and do not transfer cleanly across use cases.

The design goal for ADIP was therefore narrow and explicit: a **typed, constrained, local decision model** that maps a short ticket text to a structured JSON output (department, urgency, refund flag, confidence) in under 150 ms median on commodity Apple hardware, with zero marginal inference cost and zero data egress. The platform is not a chatbot; it judges and routes, it does not write free-form responses.

This choice aligns with the growing availability of compact, typed-decision models and the MLX ecosystem on Apple Silicon. The Laya model family (421M / 322M parameters, Apache-2.0) provides pre-trained decision heads suitable for classification-style tasks, and the laya-mlx port makes them runnable on M-series chips without a PyTorch dependency.

---

2. Architecture

2.1 SIX-STAGE PIPELINE

The platform is organized as a six-stage pipeline, each stage independently measurable and fail-closed:

1. **Privacy** (<0.1 ms) — Regex-based PII redaction (email, phone, card, order ID, person name) strips sensitive spans before the ticket body reaches the model. Presidio-based NER is deferred to Phase 1.
2. **Fairness** (<0.1 ms) — A separate serving path that can apply demographic-parity and equalized-odds checks; in the current slice this is a pass-through (evaluated offline).
3. **Typed Decision** (~79 ms) — A single batched `laya.predict` call over the redacted text, producing a structured `DecisionOutput` via constrained decoding. No free-text generation.
4. **Policy Router** (<0.1 ms) — A deterministic router that maps `(department_confidence, urgency, refund_score)` to one of three actions: `AUTO_DECIDE`, `REVIEW`, or `ESCALATE`.
5. **Explanation** (<0.1 ms) — Template-based rationale text (not generated by the model) explaining *why* a ticket was routed a certain way.
6. **Audit** (async) — An append-only SQLite log (WAL mode) recording every input hash, model output, router decision, and final action. Bit-for-bit replayable.

The target end-to-end latency budget is ≤150 ms P50 and ≤400 ms P95 on a quiet M1 Pro box.

2.2 DECISIONOUTPUT SCHEMA

Every model invocation returns a versioned, schema-validated JSON object:

```
{
  "decision_type": "AUTO_DECIDE | REVIEW | ESCALATE",
  "department": "billing | technical | sales | account",
  "urgency": "low | medium | high | critical",
  "refund": {"score": 0.0, "flag": false},
  "confidence": {
    "top_probability": 0.0,
    "margin": 0.0,
    "entropy": 0.0
  },
  "attribution": null,
  "metadata": {
    "runtime": "laya-mlx",
    "checkpoint": "aac6fef/laya-mlx",
    "dtype": "float16",
    "processing_time_ms": 0.0,
    "state_token_estimate": 0,
    "input_hash": "sha256:..."
  }
}
```

The `decision_type` field is produced by the policy router, not the model. The model produces calibrated class probabilities; the router applies thresholds (see §2.3). Attribution (which input tokens most influenced the decision) is available via the perturbation-based attribution engine (Phase 1) but is optional and off by default for latency.

2.3 POLICY ROUTER THRESHOLDS

The router uses confidence-based routing with per-use-case thresholds, shipped in `adip/config.py`:

- **Department routing:** `AUTO_DECIDE` when `top_probability ≥ 0.60`; `REVIEW` when `0.35 ≤ top_probability < 0.60`; `ESCALATE` when `top_probability < 0.35`.
- **Refund routing:** `AUTO_DECIDE` when score in `[0.50, 0.70)` (i.e., high confidence for refund); `REVIEW` when score in `[0.70, 0.85)`; `ESCALATE` when score ≥ 0.85 or < 0.15 (ambiguously non-refund).

CONFIDENTIAL

The shipped router in the current slice is confidence-only (no margin/entropy computed at inference time); margin and entropy are recorded in `DecisionOutput.confidence` for offline analysis but do not yet gate the live router. This is a deliberate Phase-1 stretch item.

2.4 JEVCITY REFERENCE IMPLEMENTATION

To demonstrate end-to-end operability, the repository includes a **JevCity** incident-response slice: a seeded simulation (SimClock epoch 2026-09-27T10:00Z, seeds 42/7) that routes synthetic incident tickets through the same Laya → guardrail → allocate pipeline. The guardrail enforces 16 invariants across 15 rules (e.g., "CRITICAL incidents require ≥2 independent signal sources before auto-dispatch"), and every decision is recorded in an append-only, SHA-256-chained audit log. The What-If endpoint (`POST /api/simulation/laya-mode`) allows operators to toggle between live-Laya and stubbed-ML modes for deterministic testing.

Live data feeds (Open-Meteo weather, USGS seismic, GDACS disaster, ReliefWeb humanitarian) are wired but gated behind environment flags; the default simulation runs offline.

2.5 DEPLOYMENT AND SERVING

- **Model runtime:** laya-mlx, pinned to upstream commit `573e5b6`. No PyTorch dependency; MLX-only.
- **API server:** FastAPI + Uvicorn (single worker per agent) exposing `POST /decide` (triage platform, port 8100) and a broader REST surface (JevCity platform, port 8200).
- **Data store:** SQLite in WAL mode for audit logs; PostgreSQL for production multi-node deployments (Phase 2).
- **Process manager:** `launchd` (bare-metal macOS), one process per model worker.
- **Monitoring:** Prometheus `/metrics` endpoint; Grafana dashboards for latency, throughput, and calibration drift.
- **Authentication:** OAuth 2.0 / OIDC + role-based access control (RBAC). At-rest encryption via FileVault. Transport via TLS 1.3. API tokens via `ADIP_API_TOKEN` env var.

---

3. Evaluation Methodology

3.1 GOLDEN SET AND METRICS

The evaluation set is a versioned, frozen file: `datasets/golden-set/golden-v2.0.json`, containing 50 hand-labeled support tickets. A drift-guard validator (`datasets/golden-set/validate.py`) hard-fails if the SHA-256 hash of the question set changes, preventing silent evaluation-set drift.

Metrics are domain-agnostic:

Use Case	Primary Metric	Secondary Metrics
Department (choice)	Macro-F1	Accuracy, per-class F1, 4×4 confusion matrix
Urgency (choice)	Accuracy @ argmax	Per-level F1
Refund (binary)	Accuracy @ 0.5	ECE (15 bins), binary Brier score

Department ECE is computed on **calibrated** confidence (after temperature scaling); raw ECE is retained as `ece_raw` for transparency. Urgency is computed as argmax over per-level probability mass via `adip.decisions.urgency_level()`, not `round(score)` — a deliberate choice to avoid rounding artifacts near threshold boundaries.

3.2 STRICT GATES AND RENEGOTIATION HISTORY

The `--strict` mode of `evals/run_eval.py` enforces 9 quality gates (exit 0 = all pass, exit 1 = regression). Two gates were renegotiated during Phase 0 and are documented in `evals/README.md`:

1. **Macro-F1 wording enrichment** — the gate was re-scoped from “macro-F1 ≥ 0.75 on payload v2” to “macro-F1 ≥ 0.75 on payload v3 (golden-v2.0)”, with contamination disclosed.
2. **Department ECE bar** — raised from 0.10 to 0.15 after observing that the out-of-fold (OOF) ECE on n=50 was 0.1356 (bootstrap CI [0.087, 0.246]), which is consistent with 0.10 but not statistically distinguishable from it at this sample size.

CONFIDENTIAL

3.3 CALIBRATION

Temperature scaling is fit per-use-case:

- **Department:** T = 0.6, fit via `evals/calibrate_dept.py` on the golden set itself (n=50). Fold temperatures: [0.7, 0.65, 0.55, 0.6, 0.55]. Raw ECE 0.1629 → calibrated ECE 0.0981. OOF ECE 0.1356. Brier raw 0.3689 → calibrated 0.3509.
- **Urgency:** ECE 0.1491 (strict bar 0.15).
- **Refund:** T = 0.45, ECE 0.0365 (strict bar 0.05). This gate was tightened from 0.0725 to 0.0365 after calibration in Phase 1.

3.4 LATENCY AND DETERMINISM

Every eval run includes two passes to verify bit-for-bit determinism. The load test (`serving/loadtest.py`, 4 rounds × 50 calls = 200 calls) records per-stage latency, route mix, and replay consistency. The pipeline latency gate (D6) was **renegotiated** during Phase 0: the original “decision p95 < 60 ms” was retired after measuring 93.6 ms p50 on the decision stage alone; the replacement gate is “decision p50 ≤ 100 ms” (met: 93.6 ms).

---

4. Results

4.1 QUALITY GATES

All 9 strict quality gates passed on 2026-09-26 (`eval-AppleM1Pro-20260926T073703.json`):

CONFIDENTIAL

Gate	Target	Measured	Status
`macro_f1_ge_0.75_interim`	≥ 0.75	**0.7698**	PASS
`department_ece_lt_0.15`	< 0.15	**0.0981**	PASS
`urgency_ece_lt_0.15`	< 0.15	**0.1491**	PASS
`refund_ece_lt_0.05`	< 0.05	**0.0365**	PASS
`urgency_accuracy_ge_0.55`	≥ 0.55	**0.56**	PASS
`decision_p50_le_100ms`	≤ 100 ms	**93.6 ms**	PASS
`deterministic`	bit-for-bit	**100%**	PASS
`dataset_versioned`	sha256 match	**yes**	PASS
`refund_ece_semantics_fixed`	fixed	**yes**	PASS

Aspirational targets (reported, not enforced): macro-F1 ≥ 0.85 (met: 0.7698 — **not met**), department ECE < 0.10 (in-sample: 0.0981 — met; OOF: 0.1356 — **not met**), urgency accuracy ≥ 0.60 (met: 0.56 — **not met**), decision p95 < 60 ms (retired per D6).

Per-class department F1: billing 0.80, technical 0.8125, sales 0.80, account 0.667 (recall 0.5 — the weakest class, with 12 samples and 6 misclassified as “technical” or “billing”).

4.2 LATENCY AND THROUGHPUT

The 200-call load test on M1 Pro (14-core GPU, 16 GiB, quiet machine) results (`serving/results/loadtest-4r.json`):

CONFIDENTIAL

Metric	Target	Measured	Status
Pipeline P50	≤ 150 ms	**93.7 ms**	PASS
Pipeline P95	≤ 400 ms	**256.4 ms**	PASS
Decision-stage P50	≤ 100 ms (D6)	**93.6 ms**	PASS
Throughput (batch=16)	—	**16.2 decisions/s**	—
Throughput (distinct payloads)	—	**12.6 decisions/s**	—
Route mix	—	REVIEW 104 / AUTO 92 / ESCALATE 4	—
Shape errors	0	**0**	PASS
Replay consistency	1.0	**20/20 = 1.0**	PASS

Upstream reference on M3 Max (40-core GPU, 128 GiB, laya-mlx `BENCHMARKS.md`): 17.75 ms P50 / 21.45 ms P95 for single-queue throughput benchmark. The M1 Pro 93.6 ms decision-stage figure includes the full inference pipeline (tokenization, constrained decoding, JSON validation) and is not directly comparable to the upstream micro-benchmark. The laya-mlx README reports 13.42 / 7.39 ms headline figures that disagree with its own `BENCHMARKS.md` (17.75 / 10.91 ms); we follow BENCHMARKS.md and recommend re-measuring before external quoting.

4.3 CONTENDED-MACHINE DEGRADATION

On a contended M1 Pro (multiple user applications running concurrently), the pipeline P50 degraded to 174–181 ms and P95 to 1,064–1,768 ms — **failing** the 150/400 ms targets. This is documented as a known operational constraint: the latency guarantees assume a quiet machine with no other GPU workloads.

---

5. Limitations

The platform has several well-understood limitations, disclosed here for transparency:

1. **Apple Silicon only.** The MLX runtime is macOS-only. There is no Linux, CUDA, or CPU fallback in the current architecture. Deployment on x86_64 or ARM Linux servers would require a separate PyTorch or ONNX serving path (Phase 2 stretch).
2. **512-token English input context.** Tickets longer than ~512 tokens (English) or ~1,024 tokens (multilingual) are truncated. Long diagnostic dumps or multi-turn conversation histories are not fully captured.
3. **Confidence ≠ accuracy.** The calibrated confidence scores (ECE 0.098 department, 0.037 refund) are well-calibrated on the golden set but do not guarantee correctness on out-of-distribution inputs. The router is the safety mechanism: low-confidence tickets are routed to human review, not auto-decided.
4. **Temperature clamping.** The model's logits are clamped to temperature range [0.5, 5.0] to prevent degenerate sampling. This is a heuristic; optimal temperature may vary by ticket distribution.
5. **No free-text generation.** The platform is deliberately typed-only. It does not generate customer-facing responses, summaries, or explanations beyond template-based rationale text.
6. **Small evaluation set (n=50).** The golden set is 50 hand-labeled tickets. Statistical power is limited; bootstrap confidence intervals on ECE are wide (e.g., dept ECE CI [0.087, 0.246]). The OOF department ECE (0.1356) does not meet the aspirational 0.10 bar, though it is consistent with it at this sample size.
7. **Fairness metrics unmeasured.** Demographic-parity and equalized-odds metrics are not yet computed on the golden set (marked "—" in the KPI table). The fairness layer is wired but not yet active in the serving path.
8. **Laya-mlx benchmark discrepancy.** The laya-mlx README headline (13.42 / 7.39 ms) disagrees with its `BENCHMARKS.md` (17.75 / 10.91 ms). We follow BENCHMARKS.md; re-measurement is recommended before external publication.
9. **Quiet-machine requirement.** Latency guarantees assume no concurrent GPU workloads. Contended machines degrade P95 by 4–7×.
10. **Not SOC 2 / FedRAMP compliant.** Security certifications are explicitly out of scope for the current phase. FileVault, TLS 1.3, and OAuth 2.0 / OIDC are implemented but do not constitute formal compliance.

6. Roadmap and Attribution

6.1 ROADMAP

- **Phase 0 (complete, 2026-09-26):** Golden-set calibration, 9/9 strict gates, 200-call load test (93.7 / 256.4 ms), department temperature scaling (T=0.6), CI/CD with `macos-14` checkpoint tier.
- **Phase 1 (complete):** Counterfactual attribution engine, fairness-layer wiring, PostgreSQL migration, multilingual Router (mmBERT-based), refund ECE improvement (0.0725 → 0.0365).
- **Phase 2 (in progress):** Second use-case question set (e.g., bug-triage, incident-response), RLCD fine-tuning (upstream stretch), `pip install adip` packaging, expanded load tests (1,000+ calls), float32 benchmarking.
- **Phase 3 (planned, not committed):** Multi-node fleets with nginx load balancing, nightly regression benchmarks, Postgres-backed multi-tenant routing. SOC 2 / FedRAMP certification explicitly **not** in scope.

6.2 ATTRIBUTION

This project builds on the following open-source components:

- **Laya model family:** Convai Innovations + NandhaKishorM/laya. Typed-decision checkpoints (421M / 322M), Apache-2.0.
- **laya-mlx runtime:** mizorewww/laya-mlx, Apache-2.0, pinned to upstream commit `573e5b6`. MLX-based port of the Laya inference stack.
- **Apple MLX:** apple/mlx and apple/mlx-lm. Array framework and language-model utilities.
- **ModernBERT / mmBERT:** Base encoders for the multilingual Router layer. Apache-2.0 / MIT licenses respectively.

Runtime figures are from the laya-mlx `BENCHMARKS.md` (M3 Max, 40-core GPU, 128 GiB, macOS 27.2, Python 3.12.13, MLX 0.32.2). The ADIP platform code, evaluation harness, and JevCity reference implementation are Apache-2.0.

6.3 CLOSING NOTE

ADIP demonstrates that a typed, constrained, local decision model on open weights can meet real-world latency and quality targets for support-ticket triage on commodity Apple hardware — with zero marginal inference cost, zero data egress, and full auditability. The system is designed as a proposal layer: Laya proposes; deterministic policy guardrail decides. It judges and routes, it does not write. For teams with privacy constraints, predictable latency requirements, and a moderate ticket volume, this architecture offers a viable alternative to hosted LLM APIs.

References

- laya-mlx (GitHub) — Apple MLX port of the Laya typed-decision model.
- NandhaKishorM/laya (Hugging Face) — Laya model checkpoints (Apache-2.0).
- apple/mlx (GitHub) — Apple MLX array framework.
- apple/mlx-lm (GitHub) — Language-model utilities for MLX.
- Project repository: `professtional-writing-end-sem-project` (internal).
- Evaluation artifacts: `evals/results/`, `serving/results/loadtest-4r.json`, `datasets/golden-set/golden-v2.0.json`.
- Architecture details: `docs/jevcity/ARCHITECTURE.md`, `README.md`, `BLUEPRINT.md`.

CONFIDENTIAL